Scientific Paper Classification Using Multi-Model NLP Pipelines

Tasnim Rahmatullah[1], **Zafra Musarrat Islam**[2], **Md Zubayer Hasan**[1], **Karib Fiyaz**[2]

[1]Department of Computer Science and Engineering, Brac University
[2]Department of Computer Science, Brac University

Abstract

Scientific literature is continuously expanding, making **automatic organization and categorization of research papers** increasingly important. The aim of this project is to **classify scientific papers into six closely related research categories** based on the abstracts of papers using various **Machine Learning techniques**: **Machine Learning, Computer Vision and Pattern Recognition, Computation and Language, Artificial Intelligence, Machine Learning (Statistics), and Neural and Evolutionary Computing**. The models evaluated are **Random Forest, Logistic Regression, Multinomial Naive Bayes, SimpleRNN, GRU, LSTM, their bidirectional counterparts, and BERT-base-uncased**, where the hyperparameters are tuned manually for each model. The top-performing model achieved an **accuracy** of **76.02%** and an **F1** of **76.08% macro-F1 (or 75.66% weighted-F1)**, and the bottom model achieved **57.69% accuracy**. Results show that basic recurrent models (SimpleRNN) are still competitive, but the representation of **contextual Transformer (BERT) and gated recurrent architectures (GRU)** perform significantly better. Overall, the experiments illustrate how the **representation of the text and the model architecture** impact the classification of similar research categories in the scientific realm.

1 Introduction

The rapid growth of **scientific literature** has made the organization and categorization of research papers an increasingly challenging task. As the number of scientific publications grows, it is becoming increasingly difficult to manually classify papers in large repositories like **arXiv**, which contain papers from many different research fields. Such collections may be organized by research category using **automatic text classification**, which is able to classify papers into appropriate research categories based on the textual content. In this work, we explore abstracts as the major source of textual information to classify scientific papers.

Scientific-paper classification is particularly challenging when the target categories are closely related. Although different research areas can share a significant amount of technical vocabulary, research areas are not always easily distinguishable from one another in general text classification tasks. An abstract of a paper from each of these disciplines (**Machine Learning, Artificial Intelligence, Computer Vision, and Natural Language Processing**) could all mention model, training, data, learning or method. During the exploratory analysis performed in this project, it was also found that there was a high amount of overlap in vocabulary within the selected categories. This means that people have to rely on multiple word and **context cues** instead of relying on specific words.

This paper makes three contributions. We build a full **multi-class classification pipeline** to classify scientific papers according to their abstracts in six closely related research areas. Second, we compare traditional machine learning, recurrent neural network and Transformer based methods with different text representations. Finally, the impact of the **preprocessing steps and hyperparameter settings** are analyzed and some advantages and disadvantages of various methods for scientific-paper classification are discussed.

2 Related Work

Natural Language Processing (NLP) has been widely used for automatically extracting meaningful information from textual data. The previous works considered in this study cover different approaches to text processing, classification, and information extraction, and provide a useful foundation for the task addressed in this work.

The first paper, **"Article classification using natural language processing and machine learning"** [Dien et al., 2019], focuses on demonstrating how NLP and machine learning techniques can be used to process textual data and identify useful patterns automatically. The approach is relevant to our task because it shows how text can be converted into meaningful representations and subsequently processed by a machine learning model. The paper also provides a basis for understanding the importance of preprocessing and selecting appropriate models when working with natural language data.

The second paper, **"Automated Research Article Classification and Recommendation Using NLP and Machine Learning"** [Rahman et al., 2025], explores and presents a more advanced approach to processing textual information. It is especially relevant in terms of its usage of **model**, due to its ability to show how a **learned representation** can be used to enhance the analysis of text, rather than relying solely on manually designed features. The work is similar to our task in that it uses machine learning to process language data and demonstrates the ability to train a machine learning model to extract patterns that are useful for the target NLP task.

A more holistic view is given by **Deshpande et al.** [Deshpande et al., 2026] in **"An End-to-End Natural Language Processing Pipeline for Intelligent News Article Analysis and Entity Relationship Extraction."** This paper combines, more or less in a single pipeline, several NLP features which are explored in more detail in the first two works. This system extracts metadata and performs abstractive summarisation, taxonomy classification, Name Entity Recognition (NER), relationship extraction, and entity normalisation. It employs **PEGASUS** for abstractive summarisation and **spaCy** for NER, while the extracted entities and relationships can be visualised in a **Neo4j knowledge graph**. It is implemented with Flask, Celery, and a React/Vite front end, demonstrating the integration of various NLP models into a real-world solution.

The paper by **Deshpande et al.** [Deshpande et al., 2026] is more relevant to our work as they illustrate how NLP models can be used to extract structured information from unstructured text. The importance of entities like persons, organisations, and locations can be identified by their use of NER, and the connection between them is determined by relationship extraction. The paper also emphasizes the need for entity normalisation, as there may be several names or forms of the same entity. Our task benefits from these techniques as they illustrate how NLP can be used to go beyond merely categorizing text, and gain insight from documents.

The models and approaches mentioned in these works [Dien et al., 2019, Rahman et al., 2025, Deshpande et al., 2026] offer theoretical and practical foundations for our task, especially the text preprocessing, representation learning, classification, and information extraction.

3 Methodology

3.1 Dataset

3.1.1 Dataset Description

The dataset used in this project is an arXiv scientific-paper dataset containing **287,421 papers with 10 attributes**. These attributes include the paper ID, title, category, category code, publication dates, authors, first author, abstract (`summary`), and abstract word count (`summary_word_count`).

For this project, the **abstract was used as the main input text**, while the paper's category was used as the target label. An initial inspection showed that the dataset has **no missing values** and contains **146 different categories.**

Since using all 146 categories would make the classification problem highly imbalanced and difficult to manage, six categories were selected:

- Machine Learning
- Computer Vision and Pattern Recognition
- Computation and Language (Natural Language Processing)

- Artificial Intelligence
- Machine Learning (Statistics)
- Neural and Evolutionary Computing

These categories were chosen because they are sufficiently large for training and they are related to one another. This is a problem when papers from the same class may mention the same technical terms.

3.1.2 Exploratory Data Analysis

Prior to the models being trained, Exploratory Data Analysis (EDA) was applied to the data to gain insight into the data and the nature of the abstracts.

Class Distribution

The category distribution showed that the six selected classes were not equally represented. **Machine Learning** and **Computer Vision and Pattern Recognition** had considerably more papers than the smaller categories.

To understand this clearly, a class-distribution bar chart was created. The imbalance was later handled by limiting each selected category to a maximum of **12,000 papers**.

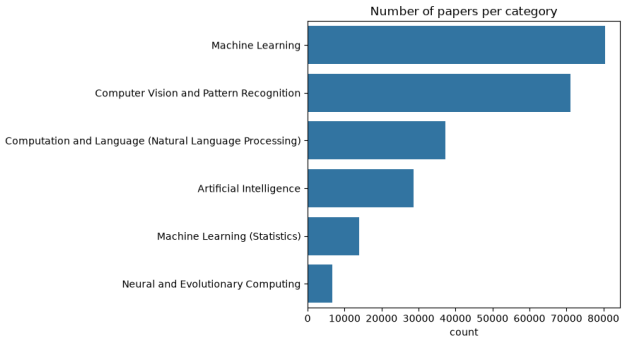


Figure 1: Number of papers per selected category before class balancing.

Abstract Length

The length of abstracts was analysed by using the word count in the `summary_word_count` column. The length of the abstracts used was within a reasonable range with the middle 50% between 140 and 200 words, but some were longer. The overall distribution of lengths was depicted as a histogram and the lengths of the abstracts were compared between the six categories using a boxplot. The choice of the maximum sequence length for neural-network models was made with the help of this analysis.

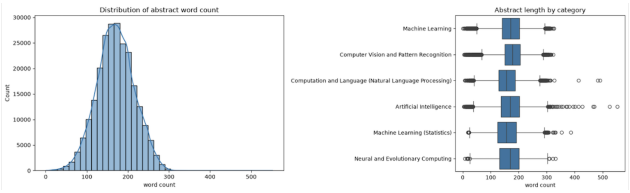


Figure 2: Abstract Length Histogram and Boxplot

Word Clouds

To gain an understanding of the vocabulary in each research area, Word Clouds were created for the 6 categories. Frequently used phrases included common words like **"model," "data," "method," and "using"** and "using" in a number of categories.

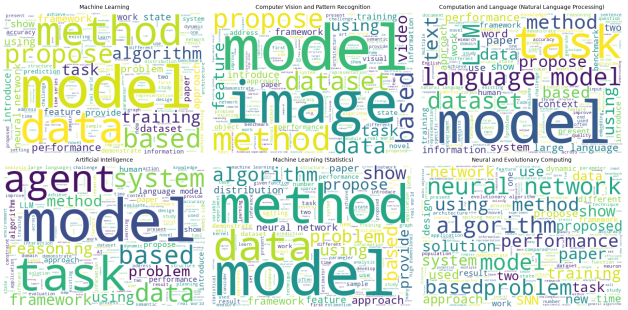


Figure 3: Word Cloud

It was found that there is an overlap in the vocabulary which indicates that the classification is not just on the basis of single words. The models should be able to read combinations of words, and know where and how they are being used.

3.2 Text Preprocessing

The abstracts were cleaned prior to being presented to the models. Due to the scientific papers included in the data set, the preprocessing also aimed at eliminating unimportant elements while retaining the information contained in the text. The following steps were used as main pre-processing steps:

1) Lowercasing: All text was converted to lowercase so that words such as "Model" and "model" were treated as the same word.

2) Removal of LaTeX: Mathematical expressions and LaTeX commands that are frequently used in

science abstracts were removed.

3) URL removal: Links that were not helpful for category prediction were eliminated from the URL.

4) Punctuation and Digit removal : Non-alphabetic characters were removed which helped to reduce the noise.

5) Tokenization: The text after cleaning was broken into individual words.

6) Stopword removal: Non-alphabetic characters were removed to reduce noise.

7) Lemmatization: Basic form of words was obtained if available. In some cases, for instance, the various forms of a word were shortened to a common basic form.

There were three different approaches to preprocessing (stopword removal with lemmatization, stopword preserving lemmatization, and stopword removal with stemming). The **stopword removal + lemmatization** method was chosen as it was useful to remove unnecessary words from the text without losing its meaning. It was not preferred to stem due to the shortened form that was generated, such as `retriev` and `augment`.

Documents with less than 10 tokens were removed because they were found to be less useful for classification because they were very short.

Abstracts were also duplicated to minimize the risk for data leakage due to the same content being present in multiple splits of the data set.

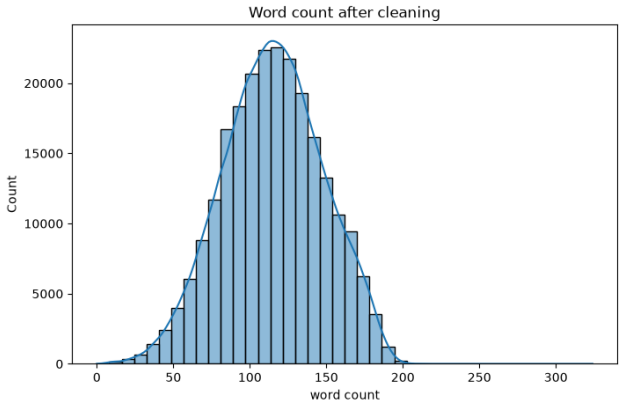


Figure 4: Distribution of abstract Word counts after Preprocessing.

3.3 Class Balancing and Data Splitting

There were different number of papers in the 6 selected categories, hence the maximum limit of 12,000 papers per category was applied. Only a random sample of papers was chosen for categories with over 12,000 papers; the smaller categories retained their papers.

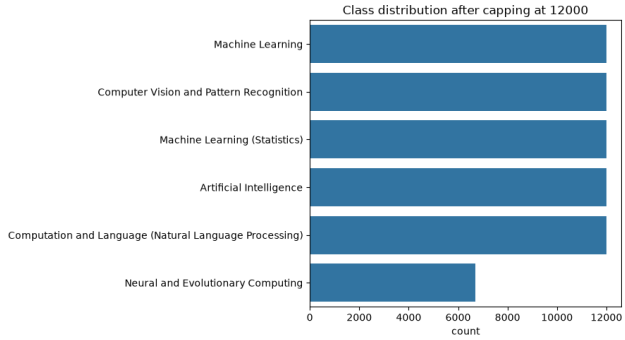


Figure 5: Class Distribution after capping each category at 12,000 papers.

The sampling was done with a fixed random state of 42 for making it reproducible. The data set was balanced and then split into 70% training, 15% validation and 15% testing data via stratified splitting.

The training set was used to train the models, the validation set was used to select the best configurations, and the test set was kept separate for the final evaluation.

3.4 Word Representations

Different text representations were used because each type captures different information from the abstracts.

TF-IDF

TF-IDF was used for the classical machine-learning models. It gives higher importance to words that are useful for a particular document but less common across the entire dataset.

The implementation used a maximum of **20,000 features** and included both **unigrams and bigrams**. Bigrams were included because combinations such as “neural network” can provide more useful information than individual words alone.

The TF-IDF vectorizer was fitted only on the training data to prevent information from the validation

and test sets from leaking into the training process.

Word2Vec
Word2Vec was used for the recurrent neural-network models. Instead of simply counting words, it represents each word as a numerical vector based on the words that appear around it.

A **200-dimensional Word2Vec model** was trained using the training corpus. A context window of 5 words, minimum word frequency of 2, Skip-gram training, and 10 epochs were used.

The resulting word vectors were used to initialize the embedding layer of the RNN-based models.

BERT
For the Transformer-based approach, **BERT-base-uncased** was used. Unlike TF-IDF and Word2Vec, BERT can represent words based on their surrounding context.

The abstracts were tokenized using the BERT tokenizer and limited to a maximum of **256 tokens**. The pretrained BERT model was then fine-tuned for the six-class classification task.

3.5 Model Architectures

The project compares **ten different model architectures**, starting with traditional machine-learning models and moving toward deep-learning and Transformer-based approaches.

Classical Machine-Learning Models
Three models were trained using TF-IDF features:

Random Forest: Multiple decision trees were combined to make the final classification. Three configurations were tested by changing the number of trees, tree depth, and minimum leaf size.

Logistic Regression: This model was used as a strong traditional baseline for text classification. Three configurations were tested by changing the regularization strength and solver.

Multinomial Naive Bayes: This probabilistic model was trained using TF-IDF features. Three different smoothing values (`alpha`) were tested.

Balanced class weighting or sample weighting was used where appropriate to reduce the effect of class imbalance.

Recurrent Neural-Network Models
Six recurrent models were implemented:

- SimpleRNN
- Gated Recurrent Unit (GRU)
- Long Short-Term Memory (LSTM)
- Bidirectional Simple RNN
- Bidirectional GRU
- Bidirectional LSTM

The cleaned abstracts were converted into integer sequences using a vocabulary of up to **30,000 words**. Each sequence was padded or truncated to **256 tokens**.

The sequences were passed through an embedding layer initialized with the 200-dimensional Word2Vec vectors. The recurrent layer was followed by dropout, a **64-unit ReLU dense layer**, and a final **six-unit softmax layer** for classification.

Three configurations were tested for each recurrent model by changing the number of recurrent units, dropout rate, and learning rate. The models were trained using the Adam optimizer for a maximum of **15 epochs** with a batch size of **128**. Early stopping was used to stop training when validation performance stopped improving.

The bidirectional models process the abstract in both forward and backward directions, allowing them to use information from both sides of the sequence.

BERT
The final model was **BERT-base-uncased**, which was fine-tuned for the six-class classification problem. Three configurations were tested using different learning rates, batch sizes, and numbers of epochs.

A batch size of **16** was used for most configurations, while a smaller batch size was used when necessary because of GPU memory limitations. With available support for the **RTX 3060 12 GB GPU**, the use of mixed-precision training was also used.

Overall, the models were designed to be a step-up from simple statistical text representations to contextual neural representations. The project was able to compare the effect on the scientific-paper classification when the different levels of understanding the text occurs.

4 Results

The results of the experiments reveal significant performance gaps between the 10 models tested across

the classic machine learning models, recurrent neural network (RNN) models and the transformer model based on BERT. The test set was used for all models with **accuracy, macro-F1 and weighted-F1 scores. Macro-F1** is very advantageous here because it allows all classes of sentiment/topic to contribute equally to the overall score, and it doesn't let the size of a particular class of sentiment/topic level be the determining factor.

Table 1: Model performance comparison ranked by test accuracy.

Rank	Model	Test Acc.	Macro-F1	Weighted-F1
1	BERT-base	76.02%	76.08%	75.66%
2	GRU	76.00%	75.99%	75.58%
3	Logistic Regression	75.39%	75.63%	75.22%
4	BiGRU	75.86%	75.25%	74.85%
5	LSTM	75.14%	75.12%	74.67%
6	BiLSTM	75.29%	74.72%	74.42%
7	Naive Bayes	74.12%	73.98%	73.59%
8	Random Forest	72.38%	71.91%	71.24%
9	BiSimpleRNN	71.82%	71.40%	71.01%
10	SimpleRNN	57.69%	55.27%	54.98%

The models showed noticeable differences in their performance. Overall, **BERT-base performed the best**, achieving **76.02% accuracy**, followed very closely by **GRU (76.00%)**. Logistic Regression also performed well with **75.39% accuracy**, showing that a simpler machine-learning approach can still be competitive. Among the RNN models, GRU performed the strongest, while **SimpleRNN had the lowest performance at 57.69% accuracy**.

4.1 BERT-base — Best Performing Model

There were significant differences between the models in their performances. Overall, **BERT-base** was the one that did the best with an accuracy of **76.02%**, closely followed by **GRU** with an accuracy of **76.00%**. Even a simpler machine-learning technique, **Logistic Regression**, performed well with **75.39% accuracy**, showing that it can still be competitive. Among the RNN models, GRU performed the strongest, while **SimpleRNN had the lowest performance at 57.69% accuracy**.

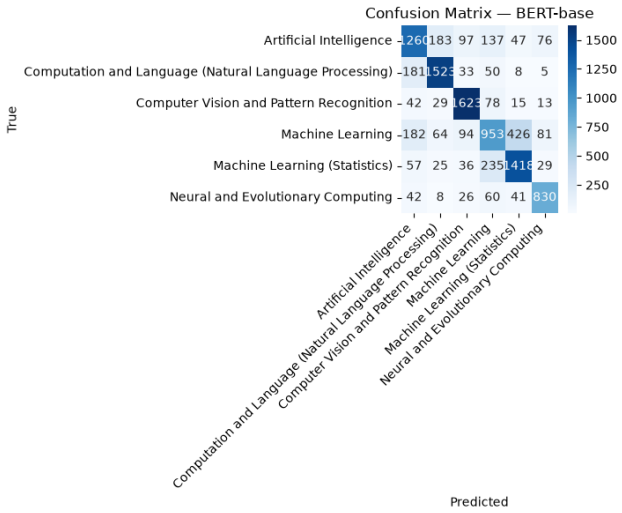


Figure 6: BERT Confusion Matrix

4.2 Gated Recurrent Unit (GRU)

The **GRU** model has nearly the same accuracy (**76.00%**) and macro-F1 (**75.99%**). This results in GRU being the strongest recurrent model in the experiment, and a very competitive alternative to BERT.

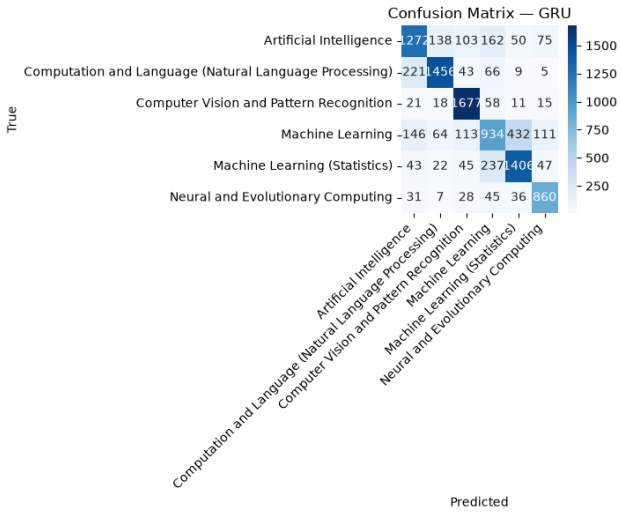


Figure 7: GRU Confusion Matrix

4.3 Logistic Regression

Logistic Regression was able to achieve a **75.39% accuracy and 75.63% macro-F1**. This was found to be good, but not better than the standard GRU model.

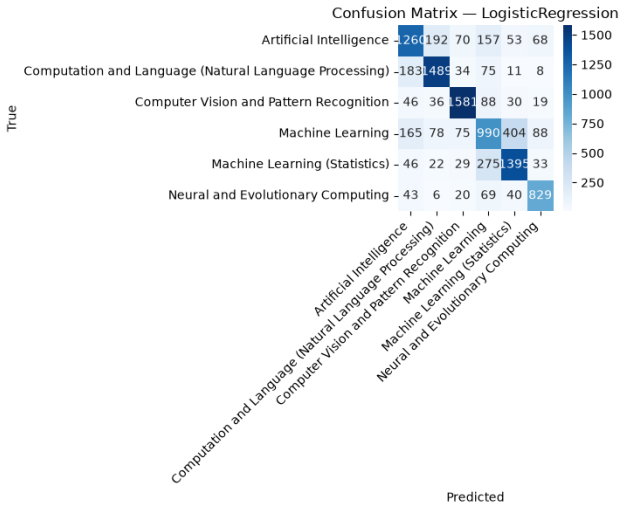


Figure 8: Logistic Regression Confusion Matrix

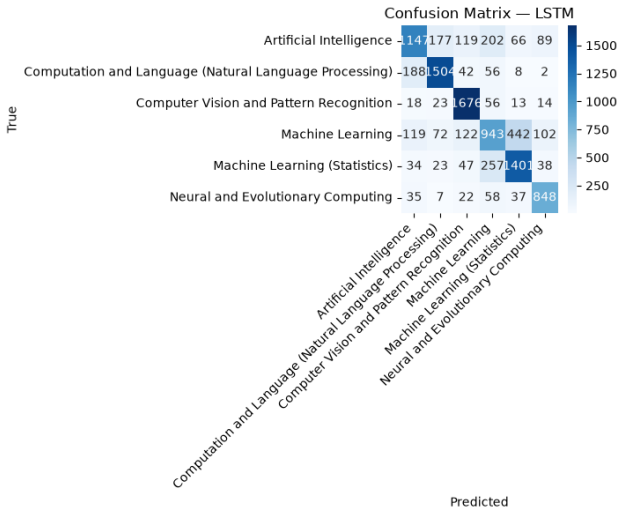


Figure 10: LSTM Confusion Matrix

4.4 BiGRU

BiGRU achieved **75.86% accuracy with a 75.25% macro-F1**. Its performance was strong, although it did not improve upon the standard GRU model.

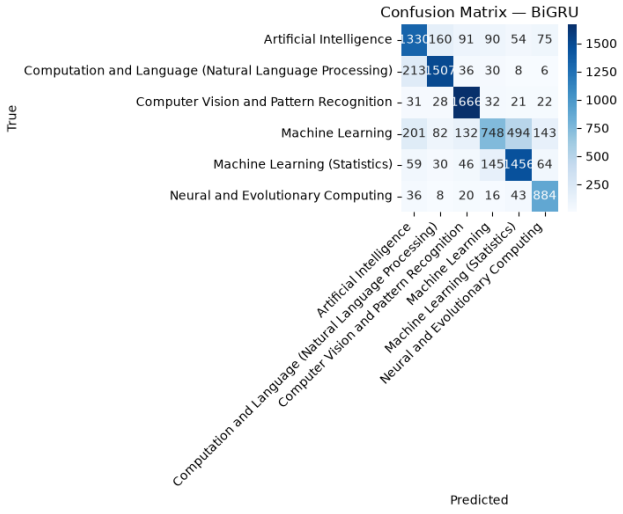


Figure 9: BiGRU Confusion Matrix

4.5 LSTM

LSTM obtained **75.14% accuracy and 75.12% macro-F1**. It provided reasonably consistent results but remained slightly behind GRU and BiGRU.

4.6 BiLSTM

BiLSTM achieved **75.29% accuracy and 74.72% macro-F1**. Its results were similar to LSTM, showing that bidirectional processing did not provide a major improvement in this experiment.

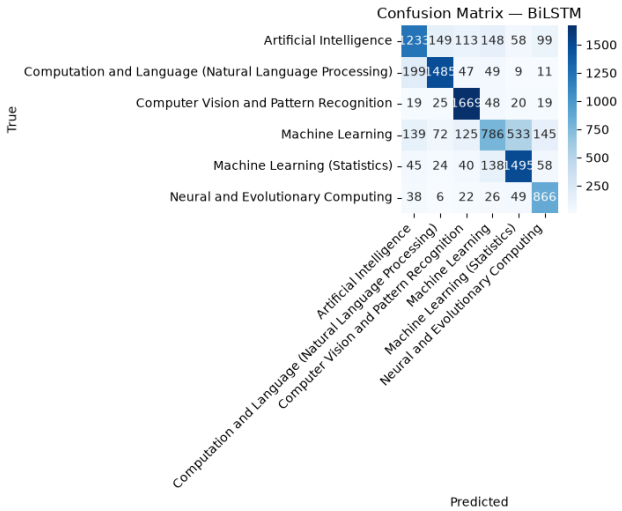


Figure 11: BiLSTM Confusion Matrix

4.7 Naive Bayes

Naive Bayes achieved an accuracy of **74.12%** and macro-F1 of **73.98%**. It wasn't as strong as the other models, but it was still a good reference point.

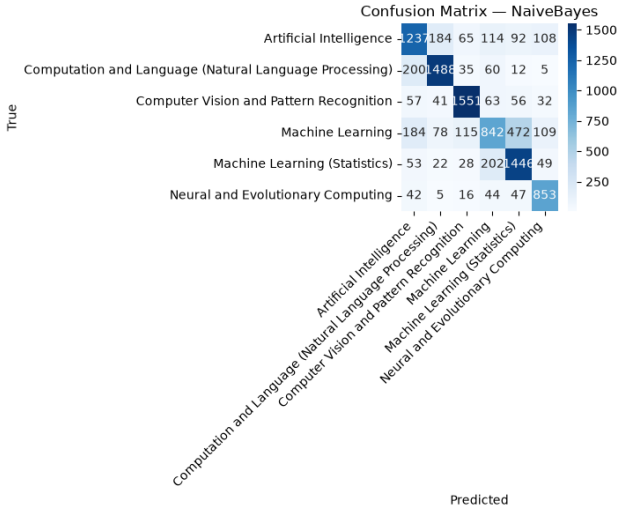


Figure 12: Naive Bayes Confusion Matrix

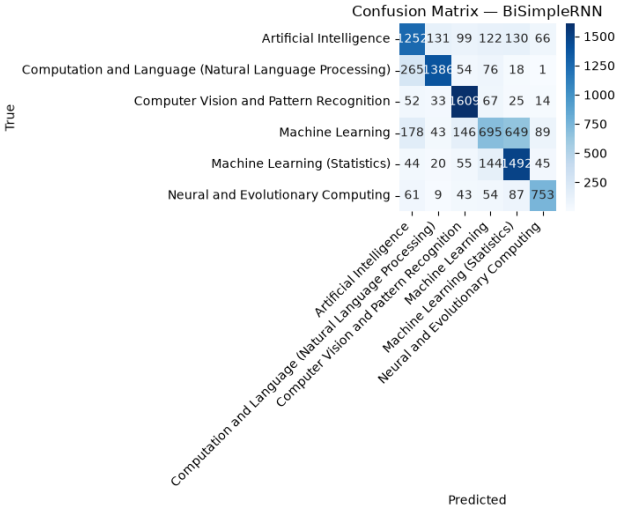


Figure 14: BiSimpleRNN Confusion Matrix

4.8 Random Forest

Random Forest achieved an accuracy of **72.38%** and macro-F1 of **71.91%**. It was also found to be more effective than the SimpleRNN with lower performance than the other recurrent architectures.

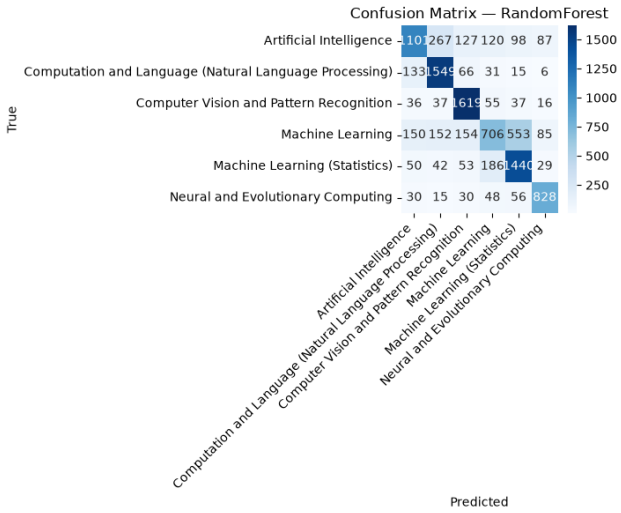


Figure 13: Random Forest Confusion Matrix

4.9 BiSimpleRNN

BiSimpleRNN achieved **71.82% accuracy and 71.40% macro-F1**. It performed better than the basic SimpleRNN but remained below the other recurrent architectures.

4.10 SimpleRNN

SimpleRNN had the lowest performance, with only **57.69% accuracy and 55.27% macro-F1**. The large gap between SimpleRNN and the other models shows that the basic recurrent architecture was not well suited to this classification task.

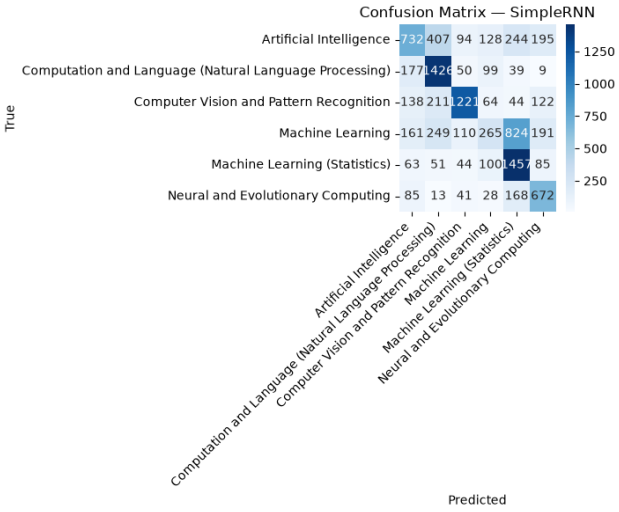


Figure 15: SimpleRNN Confusion Matrix

Overall Result

BERT-base outperformed all the other models in terms of **accuracy (76.02%), macro-F1 (76.08%), and weighted-F1 (75.66%)**. The model's performance indicates that it was able to successfully dis-

tinguish between the classes, likely through the use of information from the context in which the sentences appeared.

Using multiple confusion matrices makes the comparison more meaningful because they show not only the overall accuracy but also which classes each model classified correctly and **which classes were commonly confused**. This helps explain why some models achieved better overall scores than others.

5 Conclusion

This work compared the performance of traditional machine learning, recurrent neural network, and Transformer-based architectures for the classification of scientific paper abstracts within six closely related research areas. The results reveal the critical role of text representation and model architecture in the context of a domain-specific dataset with high vocabulary overlap.

Key Findings

The optimal overall performance was for fine-tuned **BERT-base**, with a **test accuracy of 76.02%** and a **macro-F1 score of 76.08%**, closely followed by the **GRU** model with a test accuracy of **76.00%**. Interestingly, classical **Logistic Regression** with **TF-IDF** showed outstanding performance as a low-complexity baseline model, achieving **75.39% accuracy** and surpassing other complex recurrent architectures like LSTM, BiLSTM, and BiSimpleRNN. On the contrary, **SimpleRNN** showed poor performance in this task with an accuracy of **57.69%**, as it did not have the ability to capture long-range contextual dependencies.

Limitations

The major issue with model performance is the high degree of semantic similarity between the target categories, such as **Machine Learning, Artificial Intelligence, and Machine Learning (Statistics)**, which frequently use the same vocabulary, such as the term "model." Also, using only abstracts of papers rather than the actual text bodies reduces the amount of contextual information available for fine-grained class differences. Hardware constraints also limited the batch sizes used in the fine-tuning of the BERT model and restricted testing with larger pretrained models.

Future Work

Future extensions could be made to address class ambiguity and extend the classification limits further by adding:

1. Fine-tuned, domain-specific Transformer models (e.g., **SciBERT** or **BioBERT**) using larger sequence lengths.

2. Multi-modal or structural metadata inputs, including integration of full-text body sections, author networks, or citation graphs.

3. Improved class-imbalance mitigation methods and hierarchical classification schemes to increase the separation of nested fields (such as general Machine Learning from statistics-based Machine Learning).

References

Aatmaj Deshpande, Ankita Kolage, Akanksha Darekar, and Vishal Dhotre. An end-to-end natural language processing pipeline for intelligent news article analysis and entity relationship extraction. *ScienceOpen Preprints*, 2026. doi: 10.14293/PR2199.003805.v2.

Tran Thanh Dien, Bui Huu Loc, and Nguyen Thai-Nghe. Article classification using natural language processing and machine learning. In *2019 International Conference on Advanced Computing and Applications (ACOMP)*, pages 78–84, Nha Trang, Vietnam, 2019. doi: 10.1109/ACOMP.2019.00019.

Shadikur Rahman, Hasibul Karim Shanto, Umme Ayman Koana, and Syed Muhammad Danish. Automated research article classification and recommendation using NLP and machine learning, 2025.